

.NET Full Stack Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

1. Programming & Software Development Fundamentals

- Introduction to software development
- What is full-stack development?
- Frontend vs backend development
- Client-server architecture
- Three-tier architecture
- Monolithic vs distributed applications
- How web applications work
- HTTP and HTTPS
- Request-response lifecycle
- REST APIs
- JSON
- Development, testing, staging, and production environments
- IDE setup
- Visual Studio
- Visual Studio Code
- Command-line fundamentals
- Introduction to Git and GitHub
- Introduction to Microsoft .NET ecosystem

Part I — C# Programming

2. Introduction to .NET and C#

- What is .NET?
- Evolution of .NET Framework, .NET Core, and modern .NET
- .NET ecosystem
- CLR — Common Language Runtime
- CTS — Common Type System
- CLS — Common Language Specification
- Managed code
- Assemblies
- JIT compilation
- Garbage collection
- .NET SDK
- .NET Runtime
- Creating a C# application
- Console applications
- C# program structure
- Namespaces
- Main method
- Top-level statements

3. C# Fundamentals

- Variables
- Constants
- Identifiers

- Keywords
- Data types
- Value types
- Reference types
- Nullable types
- Type inference using var
- Dynamic type
- Type conversion
- Boxing
- Unboxing
- Input and output
- String interpolation
- Operators

Operators

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Bitwise operators
- Increment/decrement operators
- Null-coalescing operators
- Conditional operators

4. Control Flow in C#

Conditional Statements

- if
- if-else
- else-if
- Nested conditions
- switch
- Switch expressions
- Pattern matching

Loops

- for
- while
- do-while
- foreach
- Nested loops
- break
- continue
- return

5. Arrays, Strings & Collections

Arrays

- One-dimensional arrays
- Multidimensional arrays
- Jagged arrays
- Array methods
- Iterating arrays

Strings

- String creation
- String methods
- String immutability
- StringBuilder
- String formatting
- String interpolation
- Comparing strings

Collections

- ArrayList concepts
- List
- Dictionary
- HashSet
- Queue
- Stack
- LinkedList
- SortedDictionary
- Concurrent collections introduction

6. Methods in C#

- Defining methods
- Calling methods
- Parameters
- Return values
- Optional parameters
- Named parameters
- ref
- out
- in
- params
- Method overloading
- Expression-bodied methods
- Local functions
- Recursion
- Lambda expressions

7. Object-Oriented Programming with C#

OOP Fundamentals

- Classes
- Objects
- Fields
- Properties
- Methods
- Constructors
- Destructors
- Access modifiers

OOP Principles

- Encapsulation
- Abstraction
- Inheritance

- Polymorphism

Advanced OOP

- Method overloading
- Method overriding
- Virtual methods
- Abstract classes
- Interfaces
- Sealed classes
- Static classes
- Partial classes
- Nested classes
- Extension methods
- Composition
- Association
- Aggregation
- Dependency injection concepts

8. Advanced C# Features

- Properties
- Auto-properties
- Indexers
- Enumerations
- Structures
- Records
- Tuples
- Nullable reference types
- Generics
- Generic classes
- Generic methods
- Delegates
- Multicast delegates
- Events
- Anonymous methods
- Lambda expressions
- Expression trees introduction
- Attributes
- Reflection
- Pattern matching
- Records vs classes
- Immutable objects

9. Exception Handling

- Types of errors
- Compile-time errors
- Runtime errors
- Logical errors
- Exception hierarchy
- try
- catch
- Multiple catch blocks
- finally

- throw
- Custom exceptions
- Exception filters
- Global exception handling concepts
- Logging exceptions
- Best practices

10. File Handling & Serialization

- File class
- FileInfo
- Directory
- DirectoryInfo
- StreamReader
- StreamWriter
- FileStream
- Binary files
- Reading/writing text files
- Working with directories
- JSON serialization
- JSON deserialization
- System.Text.Json
- XML basics
- CSV processing concepts
- Configuration files

11. LINQ — Language Integrated Query

- Introduction to LINQ
- LINQ syntax
- Method syntax
- Query syntax
- Filtering
- Projection
- Sorting
- Grouping
- Joining
- Aggregation
- Where
- Select
- OrderBy
- GroupBy
- Join
- Any
- All
- First
- FirstOrDefault
- Single
- SingleOrDefault
- Count
- Sum
- Average
- Min
- Max

- Deferred execution
- Immediate execution
- LINQ to Objects
- LINQ with databases

12. Asynchronous Programming in C#

- Synchronous vs asynchronous execution
- Tasks
- Task
- Task<T>
- async
- await
- Exception handling in async code
- Parallel programming concepts
- Threads overview
- Thread safety
- CancellationToken
- Background operations
- Asynchronous APIs
- Best practices for async programming

13. Memory Management & .NET Runtime

- Stack and heap
- Value types vs reference types
- Garbage collection
- Generations
- IDisposable
- using
- Finalizers
- Memory leaks
- Resource management
- Performance basics

14. Data Structures & Algorithms with C#

Data Structures

- Arrays
- Lists
- Stack
- Queue
- Linked list
- Hash table
- Trees
- Binary trees
- Binary search trees
- Heap
- Graphs

Algorithms

- Linear search
- Binary search
- Bubble sort

- Selection sort
- Insertion sort
- Merge sort
- Quick sort
- Recursion
- Hashing
- Two-pointer technique
- Sliding window
- Basic dynamic programming

Complexity

- Time complexity
- Space complexity
- Big-O notation

Part II — Database Development

15. SQL & SQL Server

Recommended primary database:

Microsoft SQL Server

Topics:

- Introduction to databases
- DBMS
- RDBMS
- SQL Server installation
- SQL Server Management Studio
- Databases
- Tables
- Rows
- Columns
- Data types
- Keys
- Constraints

SQL Commands

- CREATE
- ALTER
- DROP
- TRUNCATE
- INSERT
- UPDATE
- DELETE
- SELECT

Queries

- WHERE
- ORDER BY
- GROUP BY
- HAVING
- DISTINCT
- Aggregate functions

- String functions
- Date functions
- Mathematical functions

16. Advanced SQL

- Inner Join
- Left Join
- Right Join
- Full Outer Join
- Cross Join
- Self Join
- Subqueries
- Correlated subqueries
- Common Table Expressions
- Views
- Indexes
- Stored procedures
- User-defined functions
- Triggers
- Transactions
- ACID properties
- Temp tables
- Window functions
- Query optimization
- Execution plans
- Database security basics

17. Database Design

- Entity Relationship diagrams
- Primary keys
- Foreign keys
- Candidate keys
- Composite keys
- Relationships
- One-to-one
- One-to-many
- Many-to-many
- Normalization
- 1NF
- 2NF
- 3NF
- Denormalization concepts
- Indexing strategies
- Database schema design

Part III — Frontend Development

18. HTML5

- HTML document structure
- Elements

- Attributes
- Headings
- Paragraphs
- Text formatting
- Links
- Images
- Lists
- Tables
- Forms
- Input controls
- Buttons
- Semantic HTML
- Audio
- Video
- iframe
- Meta tags
- Accessibility basics
- SEO-friendly HTML

19. CSS3

- CSS syntax
- Selectors
- Colors
- Typography
- Units
- Backgrounds
- Borders
- Box model
- Margin
- Padding
- Display
- Positioning
- Z-index
- Overflow

Modern Layouts

- Flexbox
- CSS Grid

Responsive Design

- Media queries
- Mobile-first design
- Responsive layouts

Advanced CSS

- Transitions
- Transformations
- Animations
- Pseudo-classes
- Pseudo-elements
- CSS variables

20. Bootstrap / Tailwind CSS

Bootstrap

- Containers
- Grid system
- Buttons
- Cards
- Forms
- Navbar
- Modals
- Alerts
- Tables
- Responsive utilities

Tailwind CSS

- Utility classes
- Responsive utilities
- Flexbox
- Grid
- Typography
- Components
- Customization

Part IV — JavaScript Development**21. JavaScript Fundamentals**

- Variables
- var
- let
- const
- Data types
- Operators
- Type conversion
- Conditional statements
- Loops
- Functions
- Arrays
- Objects
- Strings
- Date
- Math

22. Modern JavaScript — ES6+

- Arrow functions
- Template literals
- Destructuring
- Spread operator
- Rest operator
- Default parameters
- Modules
- Classes
- Inheritance
- map

- filter
- reduce
- find
- some
- every
- Optional chaining
- Nullish coalescing

23. DOM & Browser Programming

- Document Object Model
- DOM tree
- Selecting elements
- Manipulating HTML
- Manipulating CSS
- Creating/removing elements
- Events
- Event listeners
- Forms
- Form validation
- Local storage
- Session storage
- Cookies
- Browser APIs

24. Asynchronous JavaScript

- Synchronous vs asynchronous JavaScript
- Callbacks
- Promises
- Promise chaining
- async
- await
- Fetch API
- AJAX
- JSON
- Consuming APIs
- Error handling
- Event loop concepts

Part V — Modern Frontend Framework

For a .NET full-stack course, I recommend teaching **React as the primary frontend** while optionally offering an **Angular track** because both are widely paired with ASP.NET Core.

25. React Fundamentals

- Introduction to React
- SPA architecture
- React project setup
- Vite
- JSX
- Components
- Functional components

- Props
- State
- Event handling
- Conditional rendering
- Lists
- Keys
- Forms

26. React Hooks

- useState
- useEffect
- useRef
- useContext
- useReducer
- useMemo
- useCallback
- Custom hooks

27. React Routing

- React Router
- Routes
- Route
- Link
- NavLink
- Dynamic routing
- Route parameters
- Nested routes
- Protected routes
- Error pages
- 404 handling

28. React State Management

- Component state
- Lifting state
- Context API
- Redux Toolkit
- Store
- Slices
- Reducers
- Actions
- Selectors
- Async actions

Optional:

- Zustand

29. React Forms & Validation

- Controlled forms
- Uncontrolled forms
- Form validation
- React Hook Form

- Custom validators
- File uploads
- Error handling

30. React API Integration

- Fetch API
- Axios
- GET requests
- POST
- PUT
- PATCH
- DELETE
- Authentication tokens
- JWT handling
- API interceptors
- Loading states
- Error handling
- Environment variables
- Reusable service layer

Optional Frontend Track — Angular

31. Angular Fundamentals

- Angular architecture
- TypeScript fundamentals
- Angular CLI
- Components
- Templates
- Data binding
- Directives
- Pipes
- Services
- Dependency injection
- Modules
- Standalone components

32. Angular Advanced Concepts

- Routing
- Route parameters
- Reactive forms
- Template-driven forms
- Form validation
- HTTP Client
- Interceptors
- Guards
- RxJS
- Observables
- Subjects
- State management concepts
- API integration
- JWT authentication

Part VI — ASP.NET Core Backend Development

33. ASP.NET Core Fundamentals

- Introduction to ASP.NET Core
- ASP.NET Core architecture
- Creating ASP.NET Core applications
- Project structure
- Program configuration
- Dependency injection
- Middleware
- Request pipeline
- Configuration
- Environment variables
- Logging
- Static files
- Routing
- Controllers

34. MVC Architecture

- MVC pattern
- Model
- View
- Controller
- Razor views
- Razor syntax
- Layouts
- Partial views
- ViewData
- ViewBag
- TempData
- Strongly typed views
- Model binding
- Validation
- Data annotations

35. ASP.NET Core Web API

- REST architecture
- RESTful API design
- Controllers
- ApiController
- Routing
- Attribute routing
- HTTP methods
- GET
- POST
- PUT
- PATCH
- DELETE
- HTTP status codes
- Request bodies
- Query parameters

- Route parameters
- Model binding
- Validation
- API responses
- IActionResult
- ActionResult<T>

36. DTOs & API Architecture

- Entity models
- DTOs
- Request DTOs
- Response DTOs
- Mapping
- AutoMapper concepts
- Separation of concerns
- Service layer
- Repository pattern
- Interfaces
- Dependency injection
- Business logic layer
- Data access layer

Part VII — Entity Framework Core

37. Entity Framework Core Fundamentals

- Introduction to ORM
- Entity Framework Core
- DbContext
- DbSet
- Entities
- Configuration
- Connection strings
- SQL Server integration

38. EF Core Database Approaches

- Code First
- Database First
- Model First concepts
- Migrations
- Creating migrations
- Updating databases
- Seeding data
- Reverse engineering existing databases

39. EF Core Relationships

- One-to-one
- One-to-many
- Many-to-many
- Navigation properties
- Foreign keys

- Fluent API
- Data annotations
- Cascading deletes

40. Querying with EF Core

- LINQ queries
- Filtering
- Sorting
- Pagination
- Projection
- Include
- ThenInclude
- Eager loading
- Lazy loading concepts
- Explicit loading
- Tracking
- No-tracking queries
- Raw SQL
- Transactions

41. EF Core Performance

- Query optimization
- Indexing
- Avoiding N+1 queries
- Projection
- Pagination
- Tracking vs NoTracking
- Efficient loading
- Database connection management

Part VIII — Authentication & Authorization

42. ASP.NET Core Identity

- Authentication vs authorization
- ASP.NET Core Identity
- User management
- Registration
- Login
- Logout
- Password hashing
- Password reset
- Email confirmation concepts
- Roles
- Claims
- Policies

43. JWT Authentication

- JWT structure
- Header
- Payload

- Signature
- Access token
- Refresh token
- Token creation
- Token validation
- Authentication middleware
- Authorize attributes
- Role-based authorization
- Claims-based authorization

44. OAuth & OpenID Connect

- OAuth concepts
- OAuth 2.0
- OpenID Connect
- Identity providers
- Microsoft Entra ID concepts
- Google authentication concepts
- Social login
- Single Sign-On concepts

Part IX — Full-Stack Integration

45. React / Angular + ASP.NET Core Integration

Complete architecture:

React / Angular → ASP.NET Core Web API → EF Core → SQL Server

Topics:

- Frontend-backend communication
- REST APIs
- CORS
- API services
- Login
- Registration
- JWT authentication
- Refresh tokens
- Role-based UI
- Protected routes
- Authorization
- Error handling
- Form submission
- File uploads
- Search
- Filtering
- Sorting
- Pagination
- Notifications
- Loading states

46. REST API Design Best Practices

- REST principles

- Resource naming
- HTTP verbs
- Status codes
- API validation
- DTO design
- Pagination
- Filtering
- Sorting
- Searching
- Versioning
- Swagger/OpenAPI
- Error response standards
- API documentation
- Idempotency
- Rate limiting

47. Swagger & OpenAPI

- Swagger
- OpenAPI specification
- Swagger UI
- API documentation
- Testing endpoints
- Authentication in Swagger
- XML documentation
- API schemas

Part X — Software Architecture

48. Clean Architecture

- Why software architecture matters
- Layered architecture
- N-tier architecture
- Clean Architecture
- Domain layer
- Application layer
- Infrastructure layer
- Presentation/API layer
- Dependency inversion
- Separation of concerns

49. SOLID Principles

- Single Responsibility Principle
- Open/Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Practical implementation using ASP.NET Core.

50. Design Patterns

Important patterns:

- Repository Pattern
- Unit of Work
- Dependency Injection
- Factory Pattern
- Singleton Pattern
- Strategy Pattern
- Observer Pattern
- Mediator Pattern
- CQRS introduction

51. CQRS & MediatR Concepts

- Commands
- Queries
- Separation of reads and writes
- MediatR concepts
- Command handlers
- Query handlers
- Validation pipeline
- Clean Architecture integration

Part XI — Testing

52. Unit Testing in .NET

- Testing fundamentals
- Unit tests
- Integration tests
- End-to-end tests
- AAA pattern

Frameworks:

- xUnit
- NUnit concepts
- MSTest concepts

Topics:

- Assertions
- Test fixtures
- Parameterized tests
- Mocking
- Moq
- Testing services
- Testing controllers
- Testing repositories

53. API Testing

- Postman
- Swagger
- API collections
- Environment variables

- Authentication testing
- Integration testing
- Automated API testing concepts

54. Frontend Testing

React:

- React Testing Library
- Component testing
- User interaction testing
- API mocking concepts

Angular:

- Angular testing fundamentals
- Component testing
- Service testing

Part XII — Git & Development Workflow

55. Git & GitHub

- Version control
- Repository creation
- Git initialization
- Clone
- Add
- Commit
- Push
- Pull
- Fetch
- Branching
- Merging
- Rebase basics
- Merge conflicts
- Pull requests
- Code reviews
- GitHub Issues
- .gitignore
- README
- Releases
- Tags

56. Agile & Development Practices

- Agile methodology
- Scrum
- Sprint
- Product backlog
- User stories
- Acceptance criteria
- Daily standups
- Jira fundamentals
- Code reviews

- Coding standards
- Clean code
- Documentation
- Pull request workflow

Part XIII — Advanced Backend Development

57. Logging & Monitoring

- Built-in .NET logging
- Log levels
- Structured logging
- Serilog concepts
- Application logging
- Error logging
- Request logging
- Correlation IDs
- Monitoring fundamentals

58. Caching

- Why caching is required
- In-memory caching
- Distributed caching
- Redis
- Cache-aside pattern
- Cache expiration
- Cache invalidation
- Performance optimization

59. Background Processing

- Background services
- IHostedService
- BackgroundService
- Scheduled tasks
- Hosted services
- Hangfire concepts
- Queues
- Background jobs

60. Message Queues & Event-Driven Systems

Introduction to:

- Message queues
- Producers
- Consumers
- RabbitMQ
- Azure Service Bus
- Kafka concepts
- Event-driven architecture
- Asynchronous communication

61. SignalR & Real-Time Applications

- Real-time applications
- WebSockets
- SignalR
- Hubs
- Clients
- Groups
- Notifications
- Real-time dashboards
- Chat application
- React/Angular SignalR integration

Part XIV — Application Security

62. Web Application Security

- Authentication
- Authorization
- Password hashing
- HTTPS
- SSL/TLS
- SQL injection
- Cross-Site Scripting
- CSRF
- CORS
- Broken authentication
- Secure cookies
- Input validation
- Rate limiting
- Secrets management
- Security headers
- OWASP Top 10 awareness

63. Secure ASP.NET Core Development

- Authentication middleware
- Authorization policies
- Data protection
- Secrets management
- User Secrets
- Environment variables
- Azure Key Vault concepts
- Secure configuration
- Secure API endpoints

Part XV — DevOps & Deployment

64. Linux Fundamentals

- Basic Linux commands
- Files and directories
- Permissions

- Processes
- Package management
- Networking
- Environment variables
- SSH
- Application logs

65. Web Server & Hosting Concepts

- Development server
- Web server
- Kestrel
- IIS
- Reverse proxy
- Nginx concepts
- Application hosting
- Static files
- Domains
- DNS
- HTTPS

66. Deployment Environments

Understand the complete flow:

Development → Testing → Staging → Production

Topics:

- Configuration management
- Environment-specific settings
- Secrets
- Database migration
- Logging
- Monitoring
- Deployment strategies

67. Docker

- Containers
- Containers vs virtual machines
- Docker images
- Docker containers
- Dockerfile
- Ports
- Volumes
- Networks
- Environment variables
- Docker Compose

Students containerize:

- React/Angular frontend
- ASP.NET Core API
- SQL Server / PostgreSQL
- Redis

Final architecture:

Frontend Container → API Container → Database Container

68. CI/CD

- Continuous Integration
- Continuous Delivery
- Continuous Deployment
- Build pipelines
- Automated tests
- Deployment pipelines

Tools:

- GitHub Actions
- Azure DevOps Pipelines concepts

Pipeline flow:

Commit → Build → Test → Docker Image → Deploy

Part XVI — Microsoft Azure

69. Azure Fundamentals

- Cloud computing
- IaaS
- PaaS
- SaaS
- Azure Portal
- Resource groups
- Regions
- Subscriptions
- Identity concepts

70. Deploying .NET Applications to Azure

- Azure App Service
- Deploying ASP.NET Core
- Azure Static Web Apps concepts
- Azure SQL Database
- Azure Storage
- Azure Blob Storage
- Azure Container Apps concepts
- Azure Container Registry
- Environment variables
- Application configuration

71. Azure Identity & Security

- Microsoft Entra ID
- Authentication concepts
- Managed identities
- Azure Key Vault
- Secret management
- Role-Based Access Control
- App registration concepts

72. Azure Monitoring

- Application Insights
- Azure Monitor
- Logging
- Metrics
- Traces
- Alerts
- Application health monitoring

Part XVII — Microservices Fundamentals

73. Microservices Architecture

- Monolith vs microservices
- Service decomposition
- Independent deployment
- API communication
- Database-per-service concept
- Synchronous communication
- Asynchronous communication
- Service discovery concepts
- API Gateway concepts

74. Microservices with ASP.NET Core

- Creating independent services
- REST communication
- HttpClient
- HttpClientFactory
- API Gateway concepts
- YARP concepts
- Service-to-service communication
- Resilience patterns
- Retry
- Circuit breaker concepts

Part XVIII — AI-Assisted .NET Development

This should be a dedicated module in a modern .NET full-stack course.

75. AI-Assisted Software Development

Tools can include:

- GitHub Copilot
- ChatGPT
- Claude
- Cursor
- AI coding agents

Topics:

- AI-assisted C# development
- Generating ASP.NET Core APIs

- Generating React/Angular components
- Generating SQL queries
- Explaining legacy code
- Refactoring
- Debugging
- Unit-test generation
- Documentation generation
- API documentation
- Code review with AI
- Prompting for software development
- Context engineering
- Using coding agents
- Security risks of generated code
- Validating AI-generated code
- Avoiding blind copy/paste development

76. Generative AI Integration with .NET

- Introduction to Generative AI
- LLM fundamentals
- Calling LLM APIs from C#
- Prompt engineering
- System and user prompts
- Structured output
- JSON responses
- Chat application development
- Streaming responses
- Function/tool calling concepts
- Embeddings concepts
- Vector databases introduction
- RAG introduction

Possible technologies:

- Azure OpenAI
- OpenAI APIs
- Microsoft.Extensions.AI concepts
- Semantic Kernel concepts

Example architecture:

React → ASP.NET Core → LLM → Database / Knowledge Base

77. AI-Powered Full-Stack Applications

Students can develop:

- AI chatbot
- AI customer-support portal
- Document Q&A application
- Resume analyzer
- AI content generator
- AI recommendation system
- AI-assisted search application
- Knowledge-base assistant

Part XIX — Projects

78. C# Mini Projects

Examples:

- Calculator
- Student management system
- Bank management system
- Library management system
- Employee management system
- Inventory management application

79. SQL Projects

- Employee database
- Banking database
- E-commerce database
- Student management database
- Inventory database

Students should practice:

- Schema creation
- Joins
- Views
- Stored procedures
- Functions
- Indexes
- Transactions

80. Frontend Mini Projects

- Responsive portfolio
- Product landing page
- Admin dashboard
- Task-management interface
- API-based weather application
- E-commerce UI

81. ASP.NET Core API Projects

Examples:

- Student Management API
- Employee Management API
- Product Management API
- Blog API
- Authentication API
- E-commerce API

82. Full-Stack Project 1 — Task Management System

Technology:

React + ASP.NET Core Web API + EF Core + SQL Server

Features:

- Registration
- Login
- JWT authentication
- User dashboard
- Tasks
- Categories
- Priority
- Status
- Search
- Filtering
- Pagination
- Role-based authorization

83. Full-Stack Project 2 — E-Commerce Application

Features:

- User registration
- Authentication
- Product catalog
- Categories
- Search
- Filtering
- Shopping cart
- Wishlist
- Orders
- Product images
- Admin dashboard
- Role management
- Payment gateway concepts
- REST APIs

84. Full-Stack Project 3 — Job Portal

Features:

- Candidate registration
- Employer registration
- Authentication
- Job posting
- Job searching
- Job applications
- Resume upload
- Employer dashboard
- Candidate dashboard
- Role-based access
- Notifications

85. Capstone Project

Students should complete one industry-style project following the complete SDLC:

Requirement Analysis → UI Design → Database Design → Backend → APIs → Frontend → Authentication → Testing → Docker → CI/CD → Cloud Deployment

Possible projects:

- Learning Management System
- CRM
- Hospital Management System
- Recruitment Portal
- Banking application
- E-commerce system
- Insurance application
- Project management system
- Hotel booking application
- AI-powered customer-support application

Part XX — Career & Placement Preparation

86. GitHub Profile Building

- Professional GitHub profile
- Profile README
- Repository organization
- Meaningful commit history
- Project documentation
- Screenshots
- Architecture diagrams
- API documentation
- Deployment URLs
- Portfolio projects
- Open-source fundamentals

87. Resume & LinkedIn Preparation

- .NET developer resume
- .NET full-stack developer resume
- ATS-friendly resume
- Technical skills presentation
- Project descriptions
- Quantifying accomplishments
- LinkedIn optimization
- GitHub portfolio integration

88. .NET Interview Preparation

C#

- OOP
- Collections
- Generics
- Delegates
- Events
- LINQ
- Exceptions
- Async/await
- Memory management
- Garbage collection

ASP.NET Core

- Middleware
- Dependency injection
- Routing
- Controllers
- Web APIs
- Authentication
- Authorization
- Configuration
- Logging

Entity Framework Core

- DbContext
- DbSet
- Migrations
- LINQ
- Relationships
- Loading strategies
- Tracking
- Performance

SQL

- Joins
- Subqueries
- CTEs
- Views
- Stored procedures
- Indexes
- Transactions
- Normalization

React / Angular

- Components
- State
- Props
- Hooks / Services
- Routing
- Forms
- State management
- API integration

Web Fundamentals

- HTTP
- HTTPS
- REST
- Cookies
- Sessions
- JWT
- OAuth
- CORS

89. System Design Fundamentals

- Three-tier architecture
- N-tier architecture

- Clean Architecture
- Monolith vs microservices
- REST APIs
- Database design
- Caching
- Load balancing
- Message queues
- Authentication
- Horizontal scaling
- Vertical scaling
- High availability concepts

Practice systems:

- URL shortener
- E-commerce platform
- Job portal
- Learning Management System
- Notification system

90. Mock Interviews & Coding Practice

- C# coding challenges
- SQL challenges
- JavaScript coding
- React/Angular interview questions
- ASP.NET Core interview questions
- API design exercises
- Debugging exercises
- Project explanation
- System design rounds
- Technical mock interviews
- HR interviews
- Resume review

Recommended Technology Stack

- Layer — Technologies
- Programming — C#
- Platform — .NET
- Backend — ASP.NET Core
- API — ASP.NET Core Web API
- ORM — Entity Framework Core
- Frontend Fundamentals — HTML5, CSS3, JavaScript
- Frontend Framework — React / Angular
- UI Framework — Bootstrap / Tailwind CSS
- Database — SQL Server
- Optional Database — PostgreSQL
- Caching — Redis
- Real-Time — SignalR
- Authentication — ASP.NET Core Identity, JWT, OAuth/OIDC
- API Documentation — Swagger / OpenAPI
- API Testing — Postman
- Testing — xUnit, Moq, React Testing Library

- Architecture — Clean Architecture, SOLID
- Version Control — Git, GitHub
- Containers — Docker, Docker Compose
- CI/CD — GitHub Actions / Azure DevOps
- Cloud — Microsoft Azure
- Monitoring — Application Insights
- AI-Assisted Coding — GitHub Copilot, ChatGPT, Claude/Cursor
- Generative AI — Azure OpenAI / LLM APIs, Semantic Kernel concepts
- Project Management — Jira / Agile / Scrum

Recommended Learning Sequence

Programming Fundamentals → **C#** → **OOP** → **Collections** → **LINQ** → **Async Programming** → **DSA** → **SQL Server** → **HTML** → **CSS** → **JavaScript** → **React/Angular** → **ASP.NET Core** → **Web API** → **Entity Framework Core** → **Authentication** → **Full-Stack Integration** → **Clean Architecture** → **Testing** → **Git/GitHub** → **Docker** → **CI/CD** → **Azure** → **Microservices** → **AI-Assisted Development** → **GenAI Integration** → **Capstone Project** → **Interview Preparation**

Suggested Course Positioning

For Archer Infotech, I would position this as:

AI-Assisted .NET Full-Stack Development Course

with the core stack:

C# + .NET + ASP.NET Core + Web API + Entity Framework Core + SQL Server + React/Angular + Azure + Docker + Generative AI

This makes the syllabus broader than a traditional ".NET course." Students learn not only application development, but also **modern API development, cloud deployment, DevOps, software architecture, AI-assisted coding, and AI integration**—which is much closer to how a modern .NET full-stack developer is expected to work.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.